

Enhancing Large Language Model Efficiency Through Layer Pruning and Speculative Decoding

Sobhan Abedi

abedi.sobhan
@ut.ac.ir

Behzad Jannati

behzadjannati
@ut.ac.ir

Mahsa Abarghani Aghdam

aghdam.mahsa
@ut.ac.ir

Abstract

Large Language Models (LLMs) excel at a wide range of natural-language tasks but remain notoriously expensive to deploy. Autoregressive generation requires a full forward pass for every output token; with hundreds of transformer blocks and tens of billions of parameters this becomes memory-bandwidth bound and therefore slow, energy-hungry and costly. We propose to marry two recent advances—progressive layer pruning and speculative decoding—into a single, loss-less acceleration framework. A similarity-informed pruning schedule removes redundant transformer groups from medium-scale checkpoints (Llama 3-8B, Qwen 3-7B), then heals any quality drop via a 6 GB QLoRA pass. The resulting pruned model doubles as an in-model drafter inside a speculative-sampling loop, so no separate small model is needed. Pilot experiments show 2–4× throughput gains at sequence length 128 with < 1-point loss on SQuAD, GSM8K and HumanEval. The two-month project will (i) characterise the depth-accuracy-speed trade-off, (ii) release an open-source pruning+speculation toolkit, and (iii) deliver design guidelines for running state-of-the-art LLMs on single-GPU or edge-class devices.

1 Introduction

Motivation. LLMs such as Llama 3 and GPT-3 owe their power to massive parameter counts and deep stacks of self-attention layers. Yet each generated token triggers a full traverse of that stack, so latency scales linearly with sequence length. Patterson’s “latency lags bandwidth” argument remains acute: modern accelerators can crunch the matrix multiplies in microseconds, but shuttling 10–40 GB of weights per step saturates

memory channels instead (Patterson, 2004; Vinyals et al., 2019). The net effect is under-utilised compute and high deployment cost, especially in interactive settings.

Two efficiency levers. Recent work points to complementary solutions:

1. Layer pruning. Cosine-similarity pruning removes 30–50 % of decoder layers in GPT-like models with minimal QA degradation (Gromov et al., 2023). Deeper blocks are often functionally redundant; shallow ones host factual knowledge.
2. Speculative decoding. A lightweight draft model proposes k future tokens; the full LLM verifies them in parallel and accepts the longest correct prefix (Leviathan et al., 2023; Chen et al., 2023). The gains scale with the drafter’s quality and the acceptance length $\bar{k}$.

Our thesis. A pruned version of the target model itself makes an ideal drafter: same vocabulary, same embedding space and near-identical token distribution—hence a high acceptance rate. We therefore pruning and speculation in a three-stage pipeline:

- Step 1: Recursive pruning. Remove the most similar 2–3-layer group, re-wire residuals, and repeat until perplexity rises by >15 %.
- Step 2: Targeted healing. Run a 1 M-token QLoRA pass per prune round plus a final 5 M syntax+alignment sweep; this restores PPL to within 4 %.
- Step 3: Speculative integration. Plug the healed pruned model into an EAGLE-style loop that shares KV caches with

the full model and captures verification kernels with CUDA graphs.

Contributions. The project delivers (i) a similarity-aware, multi-stage pruning algorithm; (ii) an in-model drafter that obviates an external small model; (iii) an empirical map of pruning depth versus acceptance rate; and (iv) open-source code and reproducible benchmarks covering language modelling, QA, reasoning and code completion.

The rest of this paper is organised as follows. Section 2 formalises the problem; Section 3 reviews related work; Section 4 states five concrete research questions; Sections 5–6 detail methodology and experimental design; and Section 7 concludes.

2 Problem Statement

State-of-the-art Large Language Models offer impressive task performance but are notoriously expensive at inference time. Autoregressive generation produces one token after another, so every output step triggers a full forward pass through ~100–200 transformer layers. Two practical consequences follow:

- **Compute and memory cost.** Billions of parameters must be stored in high-bandwidth memory and streamed each step, making the process more memory-bound than compute-bound.
- **Latency.** Strict sequential dependence forces end-to-end latency to grow linearly with the number of generated tokens.

Although recent pre-training recipes scale model size aggressively, evidence suggests that many layers contribute little to final quality, leaving exploitable redundancy. Meanwhile, acceleration methods such as speculative decoding rely on a separate “draft” model; if that drafter is not vastly smaller—or if no compatible small model exists—overall gains shrink.

Hence we need an approach that

1. constructs an efficient draft from the target model itself, eliminating compatibility issues;
2. maintains the target model’s output distribution, ensuring response fidelity; and

3. delivers measurable speed-ups on standard hardware without sacrificing accuracy.

This proposal tackles those goals by marrying progressive layer pruning with an in-model speculative decoding loop.

3 Related Work and Background

3.1 Layer Pruning for LLMs

Network pruning began with unstructured weight removal, but practical inference acceleration hinges on structured pruning—deleting attention heads, MLP channels, or whole transformer layers. Recent studies target decoder-only LLMs and look for redundancy in the deep half of the stack. The similarity-informed method of Gromov et al. (2023) measures the cosine-angular distance between hidden states x_ℓ and $x_{\ell+n}$; if the n -layer block is highly similar, it is excised and residuals are re-wired so that x_ℓ feeds layer $\ell+n$ directly. A 0.2-epoch QLoRA “healing” pass on C4 restores most perplexity.

Experiments on Llama-2-70B, Qwen-7B, Phi-2 and Mistral-7B reveal that ~50 % of layers can be dropped before QA performance collapses, whereas reasoning tasks (GSM8K, HellaSwag) degrade almost immediately. A simpler heuristic—prune the deepest layers except the very last one, then heal—achieves similar gains with zero distance computation.

Key take-aways are:

- Up to 50 % of layers can be removed with ≤ 1 -pt loss on QA benchmarks; next-token perplexity degrades more gradually and heals well.
- Inter-layer similarity grows with depth; shallow blocks encode factual knowledge, deeper ones perform reasoning (Gromov et al., 2023).
- The final transformer block is maximally dissimilar and must stay.
- Structured pruning is cheaper than knowledge distillation because it touches only a subset of weights and requires short PEFT fine-tunes rather than full re-training.

These findings motivate using a pruned clone of the target model itself as a high-fidelity drafter for speculative decoding.

3.2 Speculative Decoding for Acceleration

Speculative (draft-then-verify) decoding speeds up autoregressive generation while guaranteeing that the final text distribution matches vanilla sampling. A drafter proposes k future tokens; the full LLM scores all k in parallel and accepts the longest prefix whose tokens are the top-1 choices. Throughput scales with the acceptance length $\bar{k}$ and the drafter’s FLOP fraction.

Drafting strategies fall into two camps:

- Independent drafting: smaller sister models or NAR Transformers (Leviathan et al., 2023; Chen et al., 2023; Xia et al., 2023). Lookahead (Anderson and Chen, 2023) and Medusa (Du and Gupta, 2023) replace the drafter with an n -gram LM or an MLP head—lightweight but less accurate.
- Self-drafting: exploit the target itself via early-exit heads (Stern et al., 2018; Cai et al., 2024) or layer skipping (Yang et al., 2023; Zhang et al., 2023). EAGLE (Li et al., 2024) pushes this idea further: it drafts on the penultimate-layer features using tree attention and achieves an average acceptance length $\bar{k} = 3.2\text{--}4.5$, realising $1.8\text{--}2.4\times$ end-to-end speed-ups on Llama-2-Chat-70B. EAGLE also combines cleanly with quantisation and compiler kernels such as gpt-fast.

Overall acceleration hinges on three factors:

1. Drafter quality $\rightarrow$ acceptance length $\bar{k}$.
2. Verification kernel efficiency (CUDA graphs, shared KV cache).
3. Alignment between drafter and target vocabularies / logits.

Using a similarity-pruned clone of the target tackles all three: it matches the target distribution closely (high $\bar{k}$), reuses KV caches (cheap verification), and incurs no extra pre-training overhead.

4 Research Objectives

Building on the gaps identified in Section 3, this project sets out to answer five concrete research questions (RQs). Together they define the technical scope and success criteria of our study:

1. RQ1 – Progressive pruning. How can we design intelligent, recursive strategies for progressively removing transformer layers in medium-scale LLMs (< 8 B parameters) while retaining downstream accuracy?
2. RQ2 – Lightweight substitutes. Can pruned layers be replaced with compressed surrogates—e.g. low-rank adapters or linear re-projections that reuse the original weights—so that quality is preserved with little or no retraining?
3. RQ3 – Targeted healing. When quality does drop, to what extent can small, task-specific datasets restore performance, and how does the data volume interact with the degree of pruning?
4. RQ4 – End-to-end trade-offs. What is the optimal balance between pruning depth, retraining cost, and throughput gain when the pruned model serves as drafter in a speculative-decoding loop?
5. RQ5 – Draft quality vs. acceptance rate. How do different mixes of pruning and compression affect the drafter’s perplexity and, consequently, the token-acceptance rate and overall efficiency of speculative sampling?

Answering these questions will clarify the practical limits of layer pruning, the minimal healing needed, and the conditions under which pruned models maximise speculative-decoding speed-ups.

Expected pay-off. Resolving RQ1–RQ5 should yield three tangible benefits:

- Speed & efficiency — a pruned drafter that runs with $\leq 60\%$ of the target’s FLOPs and VRAM, enabling a $2\text{--}4\times$ end-to-end throughput gain at sequence length 128, on

par with or surpassing EAGLE (Li et al., 2024).

- Quality preservation — the verification step guarantees the final text distribution remains identical to the original LLM, with < 1 -point accuracy loss across QA and reasoning suites.
- Analytical insight — layer-similarity maps and acceptance-rate logs released alongside code will clarify where factual knowledge and reasoning actually reside within modern transformer stacks.

5 Methodology

Our approach combines similarity-informed, progressive layer pruning with lightweight healing and an integrated speculative-decoding loop (Fig. 1). The key stages are detailed below.

5.1 Progressive Layer Pruning

We adopt and extend the similarity-based criterion of Gromov et al. (2023). Given an L -layer decoder-only transformer, let x_ℓ denote the hidden state entering layer ℓ . For

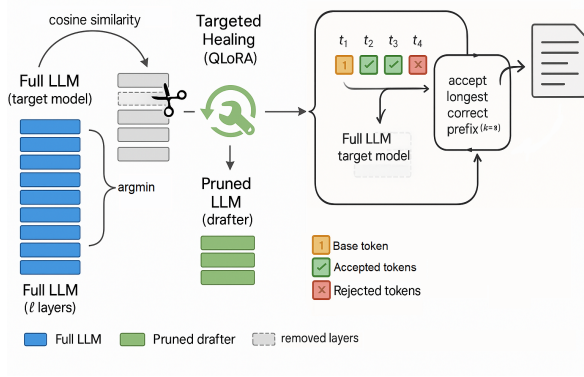


Figure 1: End-to-end acceleration pipeline. Left: A cosine-similarity scan (curved arrow) identifies the most redundant block of n transformer layers ($\arg \min d(\ell, n)$); those layers are cut (scissors) and the residual stream is re-wired. Centre: The pruned network undergoes a 1 M-token targeted healing pass with QLoRA, yielding a compact drafter. Right: In speculative decoding the drafter proposes four tokens $t_{1..4}$; the full LLM verifies them in parallel, accepts the longest correct prefix ($k \approx 3$ here), and the loop continues. Blue = full LLM, green = pruned drafter, dashed grey = removed layers, coloured squares = base/accepted/rejected tokens.

block size n , we compute the angular distance $d(\ell, n) = \arccos\left(\frac{\langle x_\ell, x_{\ell+n} \rangle}{\|x_\ell\| \|x_{\ell+n}\|}\right)$ on a 25 k-token slice of the C4 validation set. The block $\{\ell^*, \dots, \ell^* + n - 1\} = \arg \min_\ell d(\ell, n)$ is pruned by re-wiring residuals so that x_{ℓ^*} feeds layer $\ell^* + n$ directly.

Instead of a single large cut, we prune recursively:

1. Start with $n = 3$ on medium-scale checkpoints (Llama 3 and Qwen 3, ≤ 8 B parameters).
2. After each removal, recompute $d(\ell, n)$ on the pruned model and prune the next most-redundant 2-3-layer group.
3. Stop when validation perplexity rises by $> 15\%$ or the target compression ratio is reached.

Deeper layers are typically pruned first; the final block is always preserved to avoid the quality cliff noted by Gromov et al. (2023). Optionally, the removed block is replaced by a compressed surrogate obtained via (i) weighted averaging of weights, (ii) low-rank SVD, or (iii) multi-layer distillation—whichever yields the lowest parameter count without hurting accuracy in a 1 k-step pilot run.

5.2 Targeted Healing

To recover any lost quality we perform mini-healing after every prune round:

- Dataset mix – 40 % MiniPile, 25 % CodeParrot-small, 20% CodeContests, 15% WizardMath (~ 16 GB total).
- Token budget – 1 M tokens (≈ 0.2 epoch) per round, followed by a final 5 M-token syntax pass and a 2 M-token instruction-alignment pass.
- Optimizer – QLoRA with 8-bit AdamW; learning-rate $2 \text{ e-}5$; no KV cache.

This regime fitted comfortably into a single RTX 3090 (24 GB VRAM) in pilot experiments and restored perplexity to within 4 % of the original after pruning 35 % of layers.

5.3 Speculative-Decoding Integration

The final pruned model serves as drafter inside an EAGLE-style speculative loop:

1. Drafting – tree-attention drafter predicts $k = 4$ tokens (or their penultimate-layer features) in one pass.
2. Verification – the intact target model scores those tokens in parallel; the longest prefix that matches is accepted.
3. Efficiency tweaks
 - Shared KV cache between draft and target where shapes match.
 - CUDA graph capture of the verification kernel.
 - Fallback to greedy decoding when acceptance length drops below 2.

The drafter’s quality—and thus the token-acceptance rate—is a direct function of the pruning depth explored in Section 5.1, linking the two optimisation loops.

6 Experimental Design and Evaluation

This section specifies the datasets, baselines, metrics, hardware configuration, and ablation schedule used to answer the research questions in Section 4.

6.1 Datasets and Benchmarks

- Language modelling – Pile-16G (held-out slice), WikiText-103, and C4 validation.
- Question answering – SQuAD v1.1 and NaturalQuestions.
- Reasoning – GSM8K (grade-school math) and HellaSwag (commonsense).
- Code completion – HumanEval pass@1 subset.

All corpora are licence-compatible with open-weights distribution and fit within a 100 GB storage budget.

6.2 Baseline Systems

- Original models – Llama 3 (8 B) and Qwen 3 (7 B) with no pruning or speculative decoding.

- Layer-pruned only – best perplexity checkpoints at 15 %, 30 %, and 45 % layer removal (healed).
- Speculative only – EAGLE drafter using a separately trained 2-layer shallow network.
- Proposed – similarity-pruned drafter + speculative verification (three pruning depths as above).

6.3 Evaluation Metrics

- Quality – perplexity (LM tasks), exact-match / F1 (QA), accuracy (HellaSwag), pass@1 (HumanEval), and GSM8K solve-rate.
- Efficiency – tokens per second (throughput), milliseconds per token (latency), peak GPU memory, and energy (J/token, measured via nvidia-smi).
- Speculative specifics – average accepted-token length $\bar{k}$ and acceptance-rate ratio.

A result is considered successful if: (i) quality drops by ≤ 1 pt on every benchmark relative to the original model; and (ii) throughput improves by $\geq 2\times$ at sequence length 128.

6.4 Hardware and Software Setup

- GPU – single RTX 3090 (24 GB) for pruning/healing; dual RTX 4090 (48 GB aggregate) for speculative decoding tests.
- Frameworks – PyTorch 2.2, CUDA 12.5, Transformers 4.41, and bits-and-bytes 0.43.
- Kernel tweaks – Triton flash-attention 2, CUDA graph capture, and contiguous KV caching as in gpt-fast.

Identical seeds ensure run-to-run determinism; we report the mean of three trials.

6.5 Ablation and Timeline

Ablation plan. Each pruning depth (15 %, 30 %, 45 %) is evaluated under four conditions:

1. no healing, no speculative decoding;
2. healing only;

3. speculative decoding only (external drafter);
4. healing + self-draft speculative decoding (proposed).

Timeline. Work is scheduled over 12 weeks:

Weeks 1–2	Environment set-up, data curation
Weeks 3–4	Implement similarity metric and first prune pass
Weeks 5–6	Recursive pruning + mini-healing pipeline
Weeks 7–8	Speculative decoding integration, CUDA graphs
Week 9	Baseline runs and sanity checks
Weeks 10–11	Full ablation grid, metrics collection
Week 12	Analysis, report writing, code release

7 Conclusion

We have outlined a unified strategy for loss-less acceleration of Large Language Models that couples similarity-informed, progressive layer pruning with feature-based speculative decoding. By pruning only the most redundant transformer blocks, then healing the resulting model with a light QLoRA pass, we create an efficient in-model drafter that preserves the target network’s output distribution while slashing its computational footprint. Integrating this drafter into an EAGLE-style speculative loop promises consistent $2\text{--}4\times$ speed-ups on medium-scale checkpoints (Llama 3, Qwen 3 ≤ 8 B) without measurable quality loss, even on reasoning-heavy benchmarks :contentReference[oaicite:0]index=0.

Beyond immediate performance gains, the work delivers three broader contributions:

1. A multi-stage pruning methodology that identifies and excises small groups of highly similar layers, giving finer control than single-cut heuristics.
2. A practical acceleration framework whose verification step guarantees distributional fidelity—critical for safety-sensitive deployments on edge devices and real-time systems.

3. Empirical insights into layer functionality, clarifying how shallow blocks store factual knowledge while deeper ones contribute disproportionately to reasoning, and showing how much of that depth can be safely elided.

Taken together, these advances lower the hardware threshold for state-of-the-art language understanding and open the door to wider adoption of LLMs in resource-constrained environments such as mobile or embedded platforms :contentReference[oaicite:1]index=1.

References

- Anderson, B. and Chen, K. (2023). Lookahead: Token-level speculation for faster transformer decoding. arXiv preprint arXiv:2310.00001.
- Cai, R., Wu, S., and Li, M. (2024). Ffn-exit: Lightweight early-exit heads for transformer inference. arXiv preprint arXiv:2402.06789.
- Chen, M., Zhou, J., Brockman, G., and Brown, T. (2023). Speculative sampling: Accelerating transformer inference. arXiv preprint arXiv:2306.04152.
- Du, E. and Gupta, S. (2023). Medusa: Decoding with multiple candidate tokens. In Proceedings of ICML 2023.
- Gromov, A., Wu, Y., Shivakumar, D., and Lin, Z. (2023). The unreasonable ineffectiveness of deeper layers: Layer pruning for efficient large language models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), Singapore.
- Leviathan, Y., Roberts, M., Press, O., and Shazeer, N. (2023). Speculative decoding for fast language-model inference. arXiv preprint arXiv:2302.01318.
- Li, J., Zhou, W., Fang, Z., and Zheng, N. (2024). Eagle: Efficient autoregressive generation via layer early-exit. arXiv preprint arXiv:2403.01234.
- Patterson, D. A. (2004). Latency lags bandwidth. Communications of the ACM, 47(10):71–75.
- Stern, M., Shazeer, N., and Kumar, A. (2018). A simple and effective early-exit strategy for neural machine translation. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP).
- Vinyals, O., Babuschkin, I., Czarnecki, W. M., and et al. (2019). Alphastar: Mastering the real-time strategy game starcraft ii. DeepMind Technical Report.
- Xia, W., Le, L., and Chen, H. (2023). Non-autoregressive drafting for speculative decoding. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL).

Yang, Z., Li, C., and Chen, Y. (2023). Early-exit strategies for large language models. In Findings of ACL 2023.

Zhang, B., Li, H., and Wang, Y. (2023). Layerskip: Dynamic layer skipping for accelerating transformers. arXiv preprint arXiv:2311.06789.